

of speech), that *centuries* is the plural of *century*, and that *hello;* is not essentially different from *hello*. We want our chatbot to treat in the same way words that are from the same root, unless there is some reason not to.

Say we have the following query:

---

```
query = 'I want to learn about geometry algorithms.'
```

---

The first thing we can do is convert all characters to lowercase. Python's built-in `lower()` method accomplishes this:

---

```
print(query.lower())
```

---

This outputs `i want to learn about geometry algorithms..`. Another thing we can do is remove punctuation. To do that, first we'll create a Python object called a *dictionary*:

---

```
remove_punctuation_map = dict((ord(char), None) for char in string.punctuation)
```

---

This snippet creates a dictionary that maps every standard punctuation mark to the Python object `None`, and it stores the dictionary in a variable called `remove_punctuation_map`. We then use this dictionary to remove punctuation like so:

---

```
print(query.lower().translate(remove_punctuation_map))
```

---

Here, we've used the `translate()` method to take all the punctuation marks we find in the query and replace them with nothing—or in other words, remove the punctuation marks. The output we get is the same as we saw before—`i want to learn about geometry algorithms`—but without the period at the end. Next, we can perform *tokenization*, which converts a text string to a list of coherent substrings:

---

```
print(nltk.word_tokenize(query.lower().translate(remove_punctuation_map)))
```

---

We used the `nltk`'s tokenization function to accomplish this, yielding this output: `['i', 'want', 'to', 'learn', 'about', 'geometry', 'algorithms']`.

Now we can do what's called *stemming*. In English, we use the words *jump*, *jumps*, *jumping*, *jumped*, and other derived forms that are all different but share a *stem*: the verb *jump*. We don't want our chatbot to be distracted